LAPORAN
WORKSHOP Mobile Web Server
(TIF330805)
Acara 18
SEMESTER

Nama Tugas
JSON ENCODING DAN HTTP STATUS CODE (CLASS RESPONSE, VALIDASI 422, DAN RESPONSE ENVELOPE)

Nama:
Adam Ridho Saifulloh

NIM:
E41250216

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Pada Acara 17, API CRUD dibangun dengan PHP Native menggunakan arsitektur MVC, Router, dan Front Controller. Namun pembentukan response masih ditulis langsung di setiap method Controller dan pada Router, yaitu dengan memanggil http_response_code(), header(), dan json_encode() secara berulang. Akibatnya kode menjadi panjang, mudah tidak konsisten, dan format JSON yang dikirim ke client dapat berbeda-beda antara satu endpoint dengan endpoint lainnya.

Acara 18 membahas JSON encoding dan HTTP status code. Pembentukan response dipusatkan pada satu class baru, yaitu Response pada lapisan Core (app/core/Response.php), sehingga seluruh response, baik dari Router maupun Controller, memiliki format envelope yang seragam berisi success, message, data, dan timestamp. Controller kemudian direfaktor agar memanggil Response::json(), dan validasi data ditambah dengan kode status 422 (Unprocessable Entity) untuk data yang formatnya tidak memenuhi aturan.

Pada praktikum ini juga dikerjakan dua tugas praktik mandiri, yaitu menambahkan field errors pada Response::json() untuk menampung pesan kesalahan validasi per field, serta menerapkan flag JSON_PRETTY_PRINT hanya pada mode pengembangan melalui konstanta APP_ENV.

1.2 Rumusan Masalah

- Bagaimana mengubah array PHP menjadi JSON dengan json_encode() beserta flag JSON_PRETTY_PRINT, JSON_UNESCAPED_SLASHES, dan JSON_UNESCAPED_UNICODE?
- Bagaimana HTTP status code dikelompokkan dan dipilih secara tepat untuk setiap hasil pemrosesan request pada REST API?
- Bagaimana membuat class Response sebagai satu-satunya pembentuk response sehingga format JSON seragam?
- Bagaimana merefaktor Controller dan Router agar memakai Response::json() tanpa mengubah lapisan Model?
- Bagaimana menambah field errors dan mengatur pretty-print berdasarkan mode aplikasi (APP_ENV)?

1.3 Tujuan

- Memahami JSON encoding pada PHP Native dan taksonomi HTTP status code.
- Membangun class Response dan merefaktor Controller serta Router agar memakainya.
- Menerapkan validasi data dengan kode status 400, 404, 409, dan 422 beserta pesan yang informatif.
- Menguji seluruh endpoint menggunakan Postman, termasuk skenario gagal.
- Menyelesaikan tugas praktik mandiri: field errors dan JSON_PRETTY_PRINT pada mode development.

1.4 Manfaat

- Format response seragam sehingga client cukup menangani satu struktur data.
- Kode Controller lebih ringkas karena detail pembentukan JSON menjadi tanggung jawab Core.
- Kode status HTTP yang konsisten mempermudah debugging dan integrasi antar aplikasi.
- Perubahan format response cukup dilakukan pada satu tempat, yaitu class Response.

BAB II LANDASAN TEORI

2.1 JSON Encoding dan Decoding pada PHP Native

PHP menyediakan fungsi bawaan json_encode() untuk mengubah struktur data PHP (array maupun objek) menjadi teks JSON, serta json_decode() untuk proses sebaliknya. Berdasarkan dokumentasi resmi PHP Manual, json_encode() menerima flag tambahan seperti JSON_PRETTY_PRINT agar hasil diformat rapi dan mudah dibaca, JSON_UNESCAPED_SLASHES agar karakter garis miring tidak di-escape, dan JSON_UNESCAPED_UNICODE agar karakter non-ASCII ditampilkan apa adanya, bukan dalam bentuk kode unicode. Beberapa flag dapat digabung dengan operator bitwise OR (|).

2.2 Taksonomi Kode Status HTTP

HTTP status code adalah kode numerik yang diberikan server sebagai informasi hasil pemrosesan suatu request. Kode ini menjadi indikator apakah permintaan berhasil diproses atau terjadi kesalahan, dan dikelompokkan menjadi lima kategori seperti pada Tabel 2.1.

Kategori	Keterangan
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client Error
5xx	Server Error

Tabel 2.1 Kategori HTTP status code

Pada PHP Native, kode status dikirim dengan fungsi http_response_code(), misalnya http_response_code(200) atau http_response_code(404). Kode status yang sering digunakan pada REST API ditunjukkan pada Tabel 2.2.

Kode	Kelas	Contoh penggunaan
200	OK	Permintaan GET/PUT/DELETE berhasil diproses
201	Created	Data baru berhasil dibuat melalui POST
204	No Content	Berhasil tanpa mengembalikan isi
400	Bad Request	Format data pada request tidak valid
401	Unauthorized	Permintaan tidak menyertakan autentikasi yang sah
403	Forbidden	Tidak memiliki hak akses
404	Not Found	Resource atau data yang diminta tidak ditemukan
405	Method Not Allowed	Method tidak diperbolehkan
409	Conflict	Terjadi konflik data
422	Unprocessable Entity	Data valid secara format namun gagal validasi bisnis
500	Internal Server Error	Terjadi kegagalan tak terduga pada sisi server

Tabel 2.2 Status code yang sering digunakan pada REST API

Penerapan status code secara konsisten mempermudah proses debugging, meningkatkan interoperabilitas antar aplikasi, membantu client mengambil keputusan berdasarkan hasil request, dan mengikuti standar komunikasi HTTP.

2.3 Standardized Response Envelope

Praktik terbaik desain JSON API merekomendasikan format response yang konsisten, disebut response envelope, yang umumnya berisi indikator keberhasilan (success), data hasil (data), pesan (message), dan metadata tambahan seperti timestamp. Konsistensi ini memudahkan client memproses response tanpa menangani banyak variasi struktur data. Response error sebaiknya juga menyertakan rincian tiap field yang gagal validasi, yang pada praktikum ini dituangkan dalam field errors.

2.4 RFC 9457: Problem Details for HTTP APIs

RFC 9457 yang diterbitkan pada Maret 2023 adalah standar IETF yang mendefinisikan format JSON baku untuk response error HTTP API, dengan lima bidang utama yaitu type, title, status, detail, dan instance, serta tipe konten application/problem+json. Pada praktikum ini digunakan response envelope yang lebih sederhana, namun standar ini perlu dipahami karena dapat diadopsi pada API tingkat lanjut atau API publik berskala industri.

BAB III HASIL PRAKTIK

3.1 Alat dan Bahan

- Laptop/PC
- XAMPP (Apache, MySQL/MariaDB, phpMyAdmin) dengan PHP 8.2
- Visual Studio Code
- Git dan GitHub
- Web browser
- Postman
- Modul praktikum BKPM Workshop SI Web Server (TIF330805)

3.2 Tugas Praktik Mandiri

3.2.1 Tugas 1: Field errors pada Response::json()

Parameter kelima bertipe array bernama $errors ditambahkan pada Response::json() untuk menampung daftar pesan kesalahan validasi per field. Field errors hanya disertakan pada response apabila array tersebut tidak kosong, sehingga response sukses tetap ringkas. Method fail() pada BaseController juga menerima parameter $errors agar dapat diteruskan dari Controller.

File: app/core/Response.php (versi akhir)

```php
<?php
/**
 * Standarisasi seluruh response API dalam format JSON.
 */
class Response {
    public static function json(
        int $httpCode,
        bool $success,
        string $message,
        $data = null,
        array $errors = []
    ): void {
        http_response_code($httpCode);
        header('Content-Type: application/json; charset=utf-8');

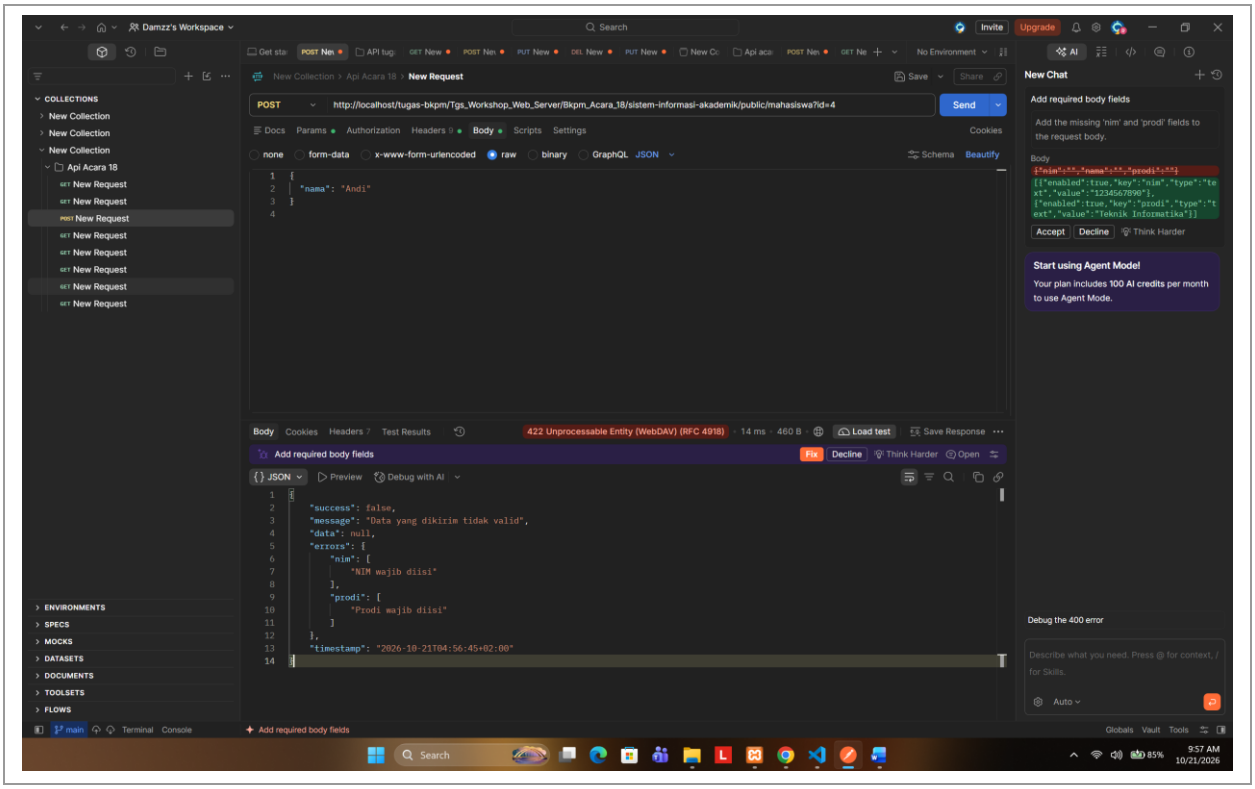
        $body = [
            'success'   => $success,
            'message'   => $message,
            'data'      => $data,
            'errors'    => (object) $errors,
            'timestamp' => date('c'),
        ];

        // JSON_PRETTY_PRINT hanya aktif di lingkungan development.
        $flags = JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE;
```

```
        if (defined('APP_ENV') && APP_ENV === 'development') {
            $flags |= JSON_PRETTY_PRINT;
        }

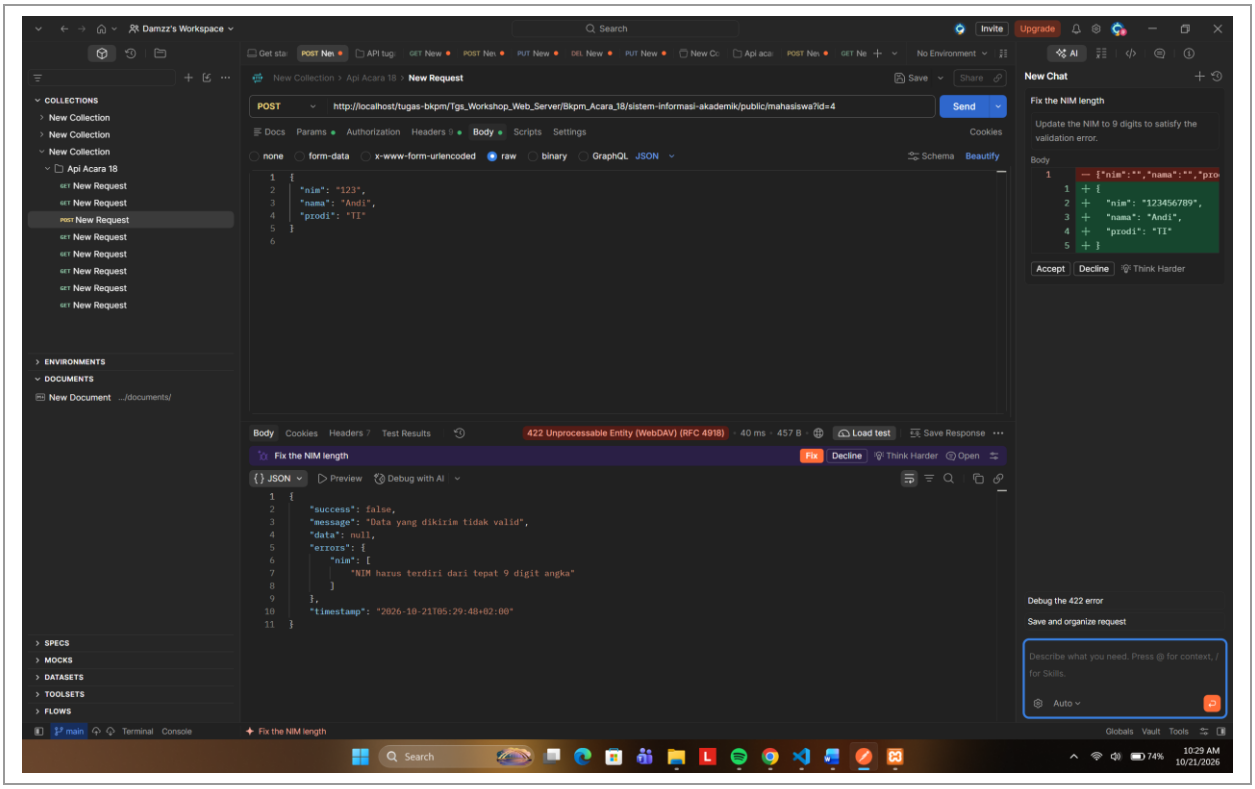
        echo json_encode($body, $flags);
        exit;
    }
}
}
```

Pada method validate(), setiap field yang tidak memenuhi aturan dicatat ke array $errors dengan nama field sebagai kunci, lalu dikirim melalui Response::json() dengan kode 400 (field kosong) atau 422 (format salah), seperti telah ditunjukkan pada Langkah 3. Contoh response ketika field kosong (400):



Gambar 3.14. Pengujian field errors pada kode 400

Contoh response ketika format NIM salah (422):



Gambar 3.15. Pengujian field errors pada kode 422

3.2.2 Tugas 2: JSON_PRETTY_PRINT Hanya pada Mode Development

Mode aplikasi disimpan pada konstanta APP_ENV di berkas konfigurasi baru app/config/config.php, yang dimuat oleh autoload.php sebelum autoloader didaftarkan. Nilai development mengaktifkan JSON_PRETTY_PRINT, sedangkan nilai production menghasilkan JSON satu baris (compact) yang lebih kecil ukurannya. Pada berkas yang sama ditambahkan pengaturan zona waktu Asia/Jakarta agar field timestamp sesuai waktu lokal (WIB, +07:00).

File: app/core/autoload.php

```php
<?php
spl_autoload_register(function ($class) {
    $folders = ['config', 'core', 'models', 'controllers'];
    foreach ($folders as $folder) {
        $path = __DIR__ . "/../{$folder}/{$class}.php";
        if (file_exists($path)) {
            require $path;
            return;
        }
    }
});
```

File: app/core/Response.php (bagian yang mengatur flag)

```php
<?php
/**
 * Standarisasi seluruh response API dalam format JSON.
 */
class Response {
    public static function json(
        int $httpCode,
        bool $success,
        string $message,
        $data = null,
        array $errors = []
    ): void {
        http_response_code($httpCode);
        header('Content-Type: application/json; charset=utf-8');

        $body = [
            'success'   => $success,
            'message'   => $message,
            'data'      => $data,
            'errors'    => (object) $errors,
            'timestamp' => date('c'),
        ];

        // JSON_PRETTY_PRINT hanya aktif di lingkungan development.
        $flags = JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE;
        if (defined('APP_ENV') && APP_ENV === 'development') {
            $flags |= JSON_PRETTY_PRINT;
        }

        echo json_encode($body, $flags);
        exit;
    }
}
```

Pengujian dilakukan pada endpoint yang sama dengan mengubah nilai APP_ENV. Hasilnya dibandingkan pada Tabel 3.2.

APP_ENV	Bentuk output JSON	Kegunaan
development	Rapi, berbaris dengan indentasi	Memudahkan membaca dan debugging
production	Satu baris, tanpa spasi tambahan	Ukuran response lebih kecil

Tabel 3.2 Perbandingan output berdasarkan APP_ENV

BAB IV KESIMPULAN

Berdasarkan praktikum Acara 18, dapat disimpulkan bahwa:

- json_encode() dengan flag JSON_UNESCAPED_SLASHES dan JSON_UNESCAPED_UNICODE menghasilkan JSON yang bersih, sedangkan JSON_PRETTY_PRINT memudahkan pembacaan hasil pada saat pengembangan.
- Class Response memusatkan pembentukan response sehingga seluruh endpoint, baik dari Router maupun Controller, memiliki envelope seragam berisi success, message, data, dan timestamp.
- Refactoring Controller dan Router agar memakai Response::json() membuat kode lebih ringkas tanpa mengubah lapisan Model.
- Penggunaan HTTP status code yang tepat (200, 201, 400, 404, 405, 409, 422, dan 500) membuat client dapat membedakan hasil request dengan jelas.
- Field errors memberi rincian kesalahan validasi per field, sedangkan konstanta APP_ENV memungkinkan pretty-print hanya aktif pada mode development.

1.